

به نام خداوند جان و خرد

آزمایش هفتم آزمایشگاه طراحی سیستم‌های دیجیتال

اعضای گروه

پوریا رحمانی ۴۰۲۱۱۱۴۱۸

امیرمحمد شربتی ۴۰۲۱۰۶۱۱۲

طراحی sender

برای طراحی، وضعیت sender را به تعدادی state تقسیم کردیم. وضعیت IDLE (شماره صفر)، وضعیتی که باید در آن سیگنال start را ارسال کند (شماره ۱)، وضعیتی که باید داده‌ها را در آن ارسال کند (شماره ۲)، وضعیتی که باید parity bit را ارسال کند (شماره ۳)، وضعیتی که باید در آن سیگنال اتمام کار را ارسال کند (شماره ۴) و وضعیتی که در آن done را یک می‌کنیم (به نشانه پایان کار) و بعد از آن قرار است به IDLE برگردیم (شماره ۵).

ورودی‌های این مدار، یک ریست active low (RstN)، یک ورودی کلاک، یک ورودی ۷ بیتی داده‌ای که باید ارسال کند و یک بیت start که با یک شدن آن روند ارسال شروع می‌شود هستند.

خروجی‌های این مدار هم یک بیتی است که باید در لحظه ارسال کند و یک بیت done که نشان می‌دهد روند ارسال به پایان رسیده است.

رجیسترهای کمکی‌ای هم برای نگهداری state، تعداد بیتی که تا کنون از داده ارسال کردیم و نگهداری داده در لحظه‌ای که start یک می‌شود استفاده می‌کنیم و از یک wire هم برای محاسبه parity bit (همان xor بیت‌های داده) استفاده می‌کنیم.

روند کار به این صورت است: اگر مدار در وضعیت reset باشد، بیت done، تعداد بیت‌هایی که از داده ارسال کردیم، بیتی که خروجی مدار است و باید ارسال شود و همینطور وضعیت مدار را (به نشانه IDLE) صفر می‌کنیم.

اگر در لبه بالا رونده کلاک، مدار در وضعیت IDLE باشد و سیگنال start هم یک باشد، باید داده ورودی مدار در لحظه را در رجیستری برای ارسال بیت به بیت ذخیره کنیم، done را صفر کنیم و state را یک کنیم تا در کلاک بعدی سیگنال شروع (که یک است) را ارسال کنیم و تعداد بیت‌های اسالی از داده را هم صفر می‌کنیم.

اگر در وضعیتی باشیم که باید سیگنال شروع ارسال را بفرستیم (وضعیت شماره یک) صرفاً خروجی مدار که روی باس قرار می‌گیرد را یک قرار می‌دهیم و برای چرخه ساعت بعدی به حالتی می‌رویم که باید داده را ارسال کنیم.

در وضعیتی که باید داده را ارسال کنیم، در هر چرخه ساعت بیتی که باید ارسال کنیم را از روی vector داده‌ای که در وضعیت صفر ذخیره کردیم و تعداد بیت‌هایی که تا کنون فرستادیم تشخیص می‌دهیم و خروجی مدار را برابر آن قرار می‌دهیم و تعداد بیت‌هایی که ارسال کردیم را یکی زیاد می‌کنیم. اگر هر ۷ بیت را ارسال کرده باشیم، به حالت بعدی می‌رویم که باید در آن بیت parity را بفرستیم.

در وضعیت ارسال بیت **parity** ، همانطور که گفته بودیم، **xor** بیت‌های داده‌ای که در وضعیت **start** ورودی مدار است (مقدار همان رجیستر کمکی که نام برده بودیم) را به یک سیم **assign** کرده بودیم. این مقدار را به عنوان بیت **parity** ارسال می‌کنیم و برای چرخه ساعت بعدی به وضعیت بعدی می‌رویم.

در وضعیت بعدی، باید سیگنال پایان (مقدار یک) را ارسال کنیم و به حالت بعدی برویم تا در چرخه ساعت بعدی سیگنال **done** را یک کنیم، مقدار **bus** (خروجی ماژول) را به صفر برگردانیم و به حالت **IDLE** برگردیم. کد به ضمیمه ارسال شده است و بنابراین از گذاشتن کد در اینجا برای جلوگیری از تکرار، اجتناب می‌کنیم.

طراحی receiver

وضعیت **reciver** را نیز به تعدادی **state** تقسیم کردیم. حالت **IDLE** (شماره صفر). از این حالت، وقتی بیت ورودی برابر یک شود، متوجه می‌شویم سیگنال شروع ارسال داده ارسال شده و در نتیجه به حالتی می‌رویم که در آن هفت بیت داده را دریافت کنیم. (شماره ۱)، پس از آن به حالتی می‌رویم که بیت **parity** را دریافت کنیم (شماره ۲)، سپس به **state** بعدی می‌رویم و **parity** کل را محاسبه می‌کنیم و سیگنال پایان را نیز دریافت می‌کنیم (شماره ۳) و در نهایت به **state** شماره ۴ می‌رویم و در آنجا **done** را یک می‌کنیم و به وضعیت **IDLE** باز می‌گردیم.

ورودی‌های این ماژول یک سیگنال کلاک، یک ریست **active low** و یک بیت ورودی به **receiver** است.

خروجی‌های این مدار، یک **vector** هشت بیتی شامل داده هفت بیتی ارسال شده به همراه **parity bit**، یک بیت نشان‌دهنده درست بودن **parity** (که یک است اگر و فقط اگر **parity** داده درست باشد، یا معادلا **xor** بیت‌های خروجی هشت بیتی ما، صفر باشد) و یک بیت **done** است که پایان عملیات ارسال را نشان می‌دهد.

از رجیسترهای کمکی برای نگهداری مقدار **state** و تعداد بیت‌هایی از داده اصلی که تا کنون دریافت شده استفاده می‌کنیم. روند کار به این صورت است:

اگر مدار در وضعیت **reset** باشد، سیگنال **done**، تعداد بیت‌های دریافتی از داده، بیت نشان‌دهنده درست بودن **parity** (صفر کردن این مورد به طور قراردادی انجام شده است)، داده خروجی و مقدار **state** (به نشانه رفتن به وضعیت **IDLE**) برابر صفر قرار می‌گیرند.

اگر در لبه بالا رونده کلاک، در وضعیت **IDLE** باشیم و بیت ورودی یک باشد، یعنی فرستنده، بیت شروع ارسال را فرستاده است. پس برای چرخه ساعت بعدی، به وضعیتی می‌رویم که باید داده را دریافت کنیم.

در وضعیتی که باید داده‌ها را دریافت کنیم، جایی که باید بیت دریافتی را ذخیره کنیم، از روی تعداد بیتی که تاکنون از داده دریافت کرده‌ایم تشخیص می‌دهیم و بعد از قرار دادن بیت در محل مناسب **vector** (که در نهایت خروجی ماژول خواهد بود) تعداد بیت‌های دریافتی را با یک جمع می‌کنیم. هر وقت هفت بیت داده را دریافت کردیم، برای دریافت **parity bit** در چرخه ساعت بعدی، به حالت بعدی می‌رویم.

در چرخه ساعت بعدی فرستنده **parity bit** را خواهد فرستاد. ما آن را دریافت کرده و در **MSB** خروجی هشت بیتی ماژول قرار می‌دهیم و برای دریافت بیت پایان ارسال و همینطور بررسی درستی **parity bit** در چرخه ساعت بعدی، وارد **state** بعدی می‌شویم.

در چرخه ساعت بعدی، مقدار خروجی `is_parity_valid` را برابر صفر بودن `xor` بیت‌های خرجی بیت‌های هشت بیتی قرار می‌دهیم و به حالت بعدی می‌رویم.

در چرخه ساعت نهایی نیز سیگنال `done` را یک می‌کنیم و به حالت `IDLE` برمی‌گردیم.

باز هم از آنجا که کد به ضمیمه ارسال شده‌است، از اینجا قرار دادن آن برای جلوگیری از تکرار خودداری می‌کنیم.

طراحی Uart

هر `Uart` یک `sender` و یک `receiver` جدا از هم دارد. غیر از یک `instance` از هر کدام از `sender` و `receiver` که در این مازول گذاشتیم، تنها نکته این بود که `baud rate` برابر 115200 است و این، به همراه این که ما یک چرخه ساعت سیگنال‌های شروع و پایان ارسال را بالا نگه می‌داریم نتیجه می‌دهد باید سیگنال کلاک‌ای تولید کنیم که هر ثانیه 115200 بار لبه مثبت چرخه ساعت دارد. ولی `clock` ما در آزمایشگاه 50 مگاهرتز است بنابراین باید از سیگنال کلاک ورودی با گذاشتن یک `counter` که هر بار $\frac{50000000}{115200 \times 2}$ می‌شمارد (حدود ۲۱۷) و سپس یک مرتبه `toggle` می‌شود، یک سیگنال ساعت بسازیم که به جای 50000000 بار، هر ثانیه 115200 بار لبه مثبت دارد.

```
reg generated_clock = 0;
reg [9:0] counter = 10'd0;

always @(posedge clk) begin
    if(counter < 217) begin
        counter <= counter + 1;
    end else begin
        counter <= 10'd 0;
        generated_clock <= ~generated_clock;
    end
end
```

طراحی تست‌بنچ

طراحی تست‌بنچ نکته خاصی نداشت. دو نمونه از `Uart` ایجاد کردیم و تک بیت خروجی `sender` اول (همان که باید روی `bus` قرار می‌داد و نه `done`) را به بیت ورودی `receiver` دومی وصل می‌کنیم و برعکس.

حال در یک سناریو، مقدار داده ورودی برای `sender` اولی را تعیین می‌کنیم و `start` آن را یک می‌کنیم. بعد از مدتی `start` آن را دوباره صفر می‌کنیم تا بعد از پایان ارسال فعلی، ارسال بعدی شروع نشود و منتظر یک شدن `done` می‌مانیم و نتیجه خروجی `receiver` دومی و درست بودن `parity` را بررسی می‌کنیم.

در سناریو بعدی نیز همزمان از اولی به دومی و از دومی به اولی ارسال می‌کنیم و منتظر یک شدن بیت `done` دریافت‌کننده هر دو می‌مانیم و نتیجه را بررسی می‌کنیم که موفقیت آمیز بود:

```
[Running] tb.v
Test 1: A is sending to B...
A started sending 1101001...
SUCCESS: B received correct data: 01101001
Parity Check: OK
-----
Test2: A and B are sending to each other simulatniously
Both UARTs started transmission...
SUCCESS: A received correct data from B: 11110000
SUCCESS: B received correct data from A: 00001111

Testbench Completed.
tb.v:145: $finish called at 223450 (1s)
[Done] exit with code=0 in 0.234 seconds
```